Practica 5

Exercici 1 - Escáner I2C

Codi

```
#include <Arduino.h>
#include <Wire.h>
void setup()
{
Wire.begin();
Serial.begin(115200);
while (!Serial);
// Leonardo: wait for serial monitor
Serial.println("\nI2C Scanner");
}
void loop()
{
byte error, address;
int nDevices;
Serial.println("Scanning...");
nDevices = 0;
for(address = 1; address < 127; address++ )
{
// The i2c_scanner uses the return value of
// the Write.endTransmisstion to see if
// a device did acknowledge to the address.
Wire.beginTransmission(address);
error = Wire.endTransmission();
if (error == 0)
{
Serial.print("I2C device found at address 0x");
if (address<16)
Serial.print("0");
Serial.print(address,HEX);
Serial.println(" !");
nDevices++;
}
else if (error==4)
{
Serial.print("Unknown error at address 0x");
if (address<16)
Serial.print("0");
Serial.println(address,HEX);
}
}
if (nDevices == 0)
Serial.println("No I2C devices found\n");
else
Serial.println("done\n");
```

```
  delay(5000);
  // wait 5 seconds for next scan
  }
```

Explicació del funcionament

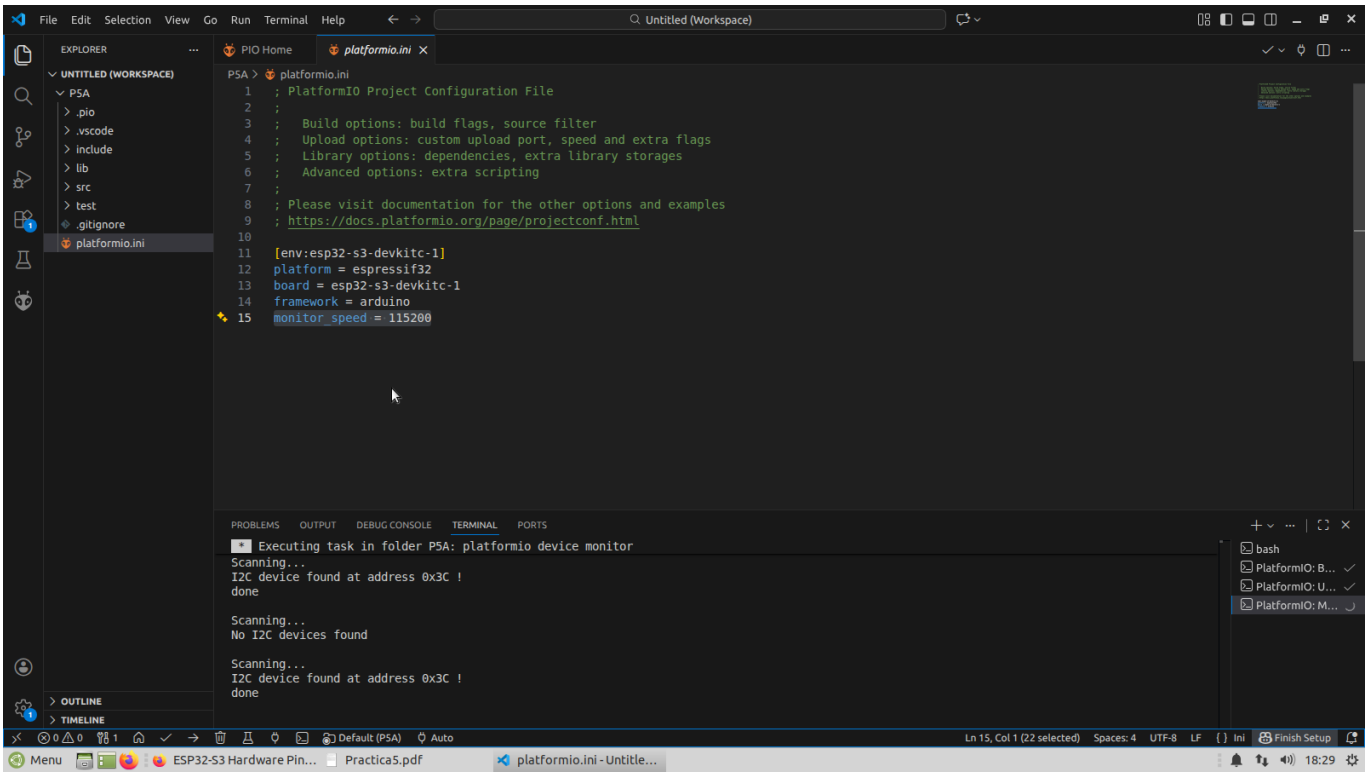
Aquest programa és un escanejador d'I2C dissenyat per identificar quins dispositius o sensors estan connectats al bus de comunicació de la teva placa. En la fase d'inicialització (setup), el codi activa la llibreria Wire per gestionar el protocol I2C i configura el port sèrie a una velocitat de 115200 bauds. També inclou una instrucció d'espera per assegurar-se que el monitor sèrie estigui llest abans de començar a mostrar els resultats de la cerca per pantalla.

Dins de la funció principal `loop`, el programa executa un bucle for que recorre sistemàticament totes les adreces possibles del protocol, que van des de la 1 fins a la 126. Per a cada adreça, s'intenta obrir un canal de comunicació mitjançant la funció Wire.beginTransmission. La part més important del codi és l'ús de `Wire.endTransmission()`, que serveix per comprovar si algun component ha respost amb un senyal de confirmació (ACK) a l'adreça consultada.

Quan el programa rep un codi de retorn igual a zero, significa que s'ha trobat un dispositiu i n'imprimeix immediatament l'adreça en format hexadecimal (per exemple, 0x3C). Si el codi d'error retornat és un quatre, el sistema informa d'un error desconegut en aquella adreça específica. Al llarg de tot el procés, el programa va sumant el nombre total de dispositius detectats per oferir un resum final al finalitzar la cerca.

Un cop s'ha completat el recorregut per totes les adreces, el programa indica si ha trobat algun component o si el bus està buit. Finalment, el codi s'atura durant cinc segons mitjançant la instrucció `delay(5000)` abans de reiniciar tot el cicle d'escaneig. Aquest funcionament periòdic és molt útil per verificar en temps real si les connexions físiques del teu projecte són correctes o si algun sensor s'ha desconnectat accidentalment.

Sortida per consola del programa


File Edit Selection View Go Run Terminal Help
Untitled (Workspace)
EXPLORER
UNTITLED (WORKSPACE)
PSA
.pio
.vscode
include
lib
src
test
.gitignore
platformio.ini
PIO Home
platformio.ini
PSA > platformio.ini
; PlatformIO Project Configuration File
;
; Build options: build flags, source filter
; Upload options: custom upload port, speed and extra flags
; Library options: dependencies, extra library storages
; Advanced options: extra scripting
;
; Please visit documentation for the other options and examples
; https://docs.platformio.org/page/projectconf.html
[env:esp32-s3-devkitc-1]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino
monitor_speed = 115200
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Executing task in folder P5A: platformio device monitor
Scanning...
I2C device found at address 0x3C !
done
Scanning...
No I2C devices found
Scanning...
I2C device found at address 0x3C !
done
bash
PlatformIO: B...
PlatformIO: U...
PlatformIO: M...
OUTLINE
TIMELINE
Default (P5A)
Auto
Ln 15, Col 1 (22 selected)
Spaces: 4
UTF-8
LF
{ } Ini
Finish Setup
Menu
ESP32-S3 Hardware Pin...
Practica5.pdf
platformio.ini - Untitle...
18:29

Exercici 2

Codi

```
/*************************************************************************
******
This is an example for our Monochrome OLEDs based on SSD1306 drivers
Pick one up today in the adafruit shop!
------> http://www.adafruit.com/category/63_98
This example is for a 128x32 pixel display using I2C to communicate
3 pins are required to interface (two I2C and one reset).
Adafruit invests time and resources providing this open
source code, please support Adafruit and open-source
hardware by purchasing products from Adafruit!
Written by Limor Fried/Ladyada for Adafruit Industries,
with contributions from the open source community.
BSD license, check license.txt for more information
All text above, and the splash screen below must be
included in any redistribution.
*************************************************************************
*****/

#include <Arduino.h>
#include <SPI.h>
#include <Wire.h>
#include <Adafruit_I2CDevice.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
void testdrawline(void); // Draw many lines
void testdrawrect(void); // Draw rectangles (outlines)
void testfillrect(void); // Draw rectangles (filled)
void testdrawcircle(void ); // Draw circles (outlines)
void testfillcircle(void ); // Draw circles (filled)
void testdrawroundrect(void ); // Draw rounded rectangles (outlines)
void testfillroundrect(void); // Draw rounded rectangles (filled)
void testdrawtriangle(void); // Draw triangles (outlines)
void testfilltriangle(void); // Draw triangles (filled)
void testdrawchar(void); // Draw characters of the default font
void testdrawstyles(void); // Draw 'stylized' characters
void testscrolltext(void); // Draw scrolling text
void testdrawbitmap(void); // Draw a small bitmap image
void testanimate(const uint8_t *bitmap, uint8_t w, uint8_t h);
#define SCREEN_WIDTH 128 // OLED display width, in pixels
#define SCREEN_HEIGHT 32 // OLED display height, in pixels
// Declaration for an SSD1306 display connected to I2C (SDA, SCL
pins)
// The pins for I2C are defined by the Wire-library.
// On an arduino UNO: A4(SDA), A5(SCL)
// On an arduino MEGA 2560: 20(SDA), 21(SCL)
// On an arduino LEONARDO: 2(SDA), 3(SCL), ...
#define OLED_RESET -1 // Reset pin # (or -1 if sharing Arduino reset
```

```
pin)
#define SCREEN_ADDRESS 0x3C ///< See datasheet for Address; 0x3D for
128x64, 0x3C for 128x32
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire,
OLED_RESET);
#define NUMFLAKES 10 // Number of snowflakes in the animation example
#define LOGO_HEIGHT 16
#define LOGO_WIDTH 16
static const unsigned char PROGMEM logo_bmp[] =
{ B00000000, B11000000,
B00000001, B11000000,
B00000001, B11000000,
B00000011, B11100000,
B11110011, B11100000,
B11111110, B11111000,
B01111110, B11111111,
B00110011, B10011111,
B00011111, B11111100,
B00001101,
B00011011,
B00111111,
B00111111,
B01111100,
B01110000,
B00000000,
B01110000,
B10100000,
B11100000,
B11110000,
B11110000,
B01110000,
B00110000 };
void setup() {
Serial.begin(115200);
// SSD1306_SWITCHCAPVCC = generate display voltage from 3.3V
internally
if(!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
Serial.println(F("SSD1306 allocation failed"));
for(;;); // Don't proceed, loop forever
}
// Show initial display buffer contents on the screen --
// the library initializes this with an Adafruit splash screen.
display.display();
delay(2000); // Pause for 2 seconds
// Clear the buffer
display.clearDisplay();
// Draw a single pixel in white
display.drawPixel(10, 10, SSD1306_WHITE);
// Show the display buffer on the screen. You MUST call display()
after
// drawing commands to make them visible on screen!
display.display();
delay(2000);
// display.display() is NOT necessary after every single drawing
```

```
  command,
  // unless that's what you want...rather, you can batch up a bunch of
  // drawing operations and then update the screen all at once by
  calling
  // display.display(). These examples demonstrate both approaches...
  testdrawline(); // Draw many lines
  testdrawrect(); // Draw rectangles (outlines)
  testfillrect(); // Draw rectangles (filled)
  testdrawcircle(); // Draw circles (outlines)
  testfillcircle(); // Draw circles (filled)
  testdrawroundrect(); // Draw rounded rectangles (outlines)
  testfillroundrect(); // Draw rounded rectangles (filled)
  testdrawtriangle(); // Draw triangles (outlines)
  testfilltriangle(); // Draw triangles (filled)
  testdrawchar(); // Draw characters of the default font
  testdrawstyles(); // Draw 'stylized' characters
  testscrolltext(); // Draw scrolling text
  testdrawbitmap(); // Draw a small bitmap image
  // Invert and restore display, pausing in-between
  display.invertDisplay(true);
  delay(1000);
  display.invertDisplay(false);
  delay(1000);
  testanimate(logo_bmp, LOGO_WIDTH, LOGO_HEIGHT); // Animate bitmaps
  }
  void loop() {
  }
  void testdrawline() {
  int16_t i;
  display.clearDisplay(); // Clear display buffer
  for(i=0; i<display.width(); i+=4) {
  display.drawLine(0, 0, i, display.height()-1, SSD1306_WHITE);
  display.display(); // Update screen with each newly-drawn line
  delay(1);
  }
  for(i=0; i<display.height(); i+=4) {
  display.drawLine(0, 0, display.width()-1, i, SSD1306_WHITE);
  display.display();
  delay(1);
  }
  delay(250);
  display.clearDisplay();
  for(i=0; i<display.width(); i+=4) {
  display.drawLine(0, display.height()-1, i, 0, SSD1306_WHITE);
  display.display();
  delay(1);
  }
  for(i=display.height()-1; i>=0; i-=4) {
  display.drawLine(0, display.height()-1, display.width()-1, i,
  SSD1306_WHITE);
  display.display();
  delay(1);
  }
  delay(250);
```

```
  display.clearDisplay();
  for(i=display.width()-1; i>=0; i-=4) {
  display.drawLine(display.width()-1, display.height()-1, i, 0,
  SSD1306_WHITE);
  display.display();
  delay(1);
  }
  for(i=display.height()-1; i>=0; i-=4) {
  display.drawLine(display.width()-1, display.height()-1, 0, i,
  SSD1306_WHITE);
  display.display();
  delay(1);
  }
  delay(250);
  display.clearDisplay();
  for(i=0; i<display.height(); i+=4) {
  display.drawLine(display.width()-1, 0, 0, i, SSD1306_WHITE);
  display.display();
  delay(1);
  }
  for(i=0; i<display.width(); i+=4) {
  display.drawLine(display.width()-1, 0, i, display.height()-1,
  SSD1306_WHITE);
  display.display();
  delay(1);
  }
  delay(2000); // Pause for 2 seconds
  }
  void testdrawrect(void) {
  display.clearDisplay();
  for(int16_t i=0; i<display.height()/2; i+=2) {
  display.drawRect(i, i, display.width()-2*i, display.height()-2*i,
  SSD1306_WHITE);
  display.display(); // Update screen with each newly-drawn rectangle
  delay(1);
  }
  delay(2000);
  }
  void testfillrect(void) {
  display.clearDisplay();
  for(int16_t i=0; i<display.height()/2; i+=3) {
  // The INVERSE color is used so rectangles alternate white/black
  display.fillRect(i, i, display.width()-i*2, display.height()-i*2,
  SSD1306_INVERSE);
  display.display(); // Update screen with each newly-drawn rectangle
  delay(1);
  }
  delay(2000);
  }
  void testdrawcircle(void) {
  display.clearDisplay();
  for(int16_t i=0; i<max(display.width(),display.height())/2; i+=2) {
  display.drawCircle(display.width()/2, display.height()/2, i,
  SSD1306_WHITE);
```

```
display.display();
delay(1);
}
delay(2000);
}
void testfillcircle(void) {
display.clearDisplay();
for(int16_t i=max(display.width(),display.height())/2; i>0; i-=3) {
// The INVERSE color is used so circles alternate white/black
display.fillCircle(display.width() / 2, display.height() / 2, i,
SSD1306_INVERSE);
display.display(); // Update screen with each newly-drawn circle
delay(1);
}
delay(2000);
}
void testdrawroundrect(void) {
display.clearDisplay();
for(int16_t i=0; i<display.height()/2-2; i+=2) {
display.drawRoundRect(i, i, display.width()-2*i,
display.height()-2*i,
display.height()/4, SSD1306_WHITE);
display.display();
delay(1);
}
delay(2000);
}
void testfillroundrect(void) {
display.clearDisplay();
for(int16_t i=0; i<display.height()/2-2; i+=2) {
// The INVERSE color is used so round-rects alternate white/black
display.fillRoundRect(i, i, display.width()-2*i,
display.height()-2*i,
display.height()/4, SSD1306_INVERSE);
display.display();
delay(1);
}
delay(2000);
}
void testdrawtriangle(void) {
display.clearDisplay();
for(int16_t i=0; i<max(display.width(),display.height())/2; i+=5) {
display.drawTriangle(
display.width()/2 , display.height()/2-i,
display.width()/2-i, display.height()/2+i,
display.width()/2+i, display.height()/2+i, SSD1306_WHITE);
display.display();
delay(1);
}
delay(2000);
}
void testfilltriangle(void) {
display.clearDisplay();
for(int16_t i=max(display.width(),display.height())/2; i>0; i-=5) {
```

```
// The INVERSE color is used so triangles alternate white/black
display.fillTriangle(
display.width()/2 , display.height()/2-i,
display.width()/2-i, display.height()/2+i,
display.width()/2+i, display.height()/2+i, SSD1306_INVERSE);
display.display();
delay(1);
}
delay(2000);
}
void testdrawchar(void) {
display.clearDisplay();
display.setTextSize(1);
// Normal 1:1 pixel scale
display.setTextColor(SSD1306_WHITE); // Draw white text
display.setCursor(0, 0);
// Start at top-left corner
display.cp437(true);
// Use full 256 char 'Code Page 437' font
// Not all the characters will fit on the display. This is normal.
// Library will draw what it can and the rest will be clipped.
for(int16_t i=0; i<256; i++) {
if(i == '\n') display.write(' ');
else
display.write(i);
}
display.display();
delay(2000);
}
void testdrawstyles(void) {
display.clearDisplay();
display.setTextSize(1);
// Normal 1:1 pixel scale
display.setTextColor(SSD1306_WHITE);
// Draw white text
display.setCursor(0,0);
// Start at top-left corner
display.println(F("Hello, world!"));
display.setTextColor(SSD1306_BLACK, SSD1306_WHITE); // Draw 'inverse'
text
display.println(3.141592);
display.setTextSize(2); // Draw 2X-scale text
display.setTextColor(SSD1306_WHITE);
display.print(F("0x")); display.println(0xDEADBEEF, HEX);
display.display();
delay(2000);
}
void testscrolltext(void) {
display.clearDisplay();
display.setTextSize(2); // Draw 2X-scale text
display.setTextColor(SSD1306_WHITE);
display.setCursor(10, 0);
display.println(F("scroll"));
display.display();
```

```
// Show initial text
delay(100);
// Scroll in various directions, pausing in-between:
display.startscrollright(0x00, 0x0F);
delay(2000);
display.stopscroll();
delay(1000);
display.startscrollleft(0x00, 0x0F);
delay(2000);
display.stopscroll();
delay(1000);
display.startscrolldiagright(0x00, 0x07);
delay(2000);
display.startscrolldiagleft(0x00, 0x07);
delay(2000);
display.stopscroll();
delay(1000);
}
void testdrawbitmap(void) {
display.clearDisplay();
display.drawBitmap(
(display.width() - LOGO_WIDTH ) / 2,
(display.height() - LOGO_HEIGHT) / 2,
logo_bmp, LOGO_WIDTH, LOGO_HEIGHT, 1);
display.display();
delay(1000);
}
#define XPOS 0 // Indexes into the 'icons' array in function below
#define YPOS 1
#define DELTAY 2
void testanimate(const uint8_t *bitmap, uint8_t w, uint8_t h) {
int8_t f, icons[NUMFLAKES][3];
// Initialize 'snowflake' positions
for(f=0; f< NUMFLAKES; f++) {
icons[f][XPOS]
= random(1 - LOGO_WIDTH, display.width());
icons[f][YPOS]
= -LOGO_HEIGHT;
icons[f][DELTAY] = random(1, 6);
Serial.print(F("x: "));
Serial.print(icons[f][XPOS], DEC);
Serial.print(F(" y: "));
Serial.print(icons[f][YPOS], DEC);
Serial.print(F(" dy: "));
Serial.println(icons[f][DELTAY], DEC);
}
for(;;) { // Loop forever...
display.clearDisplay(); // Clear the display buffer
// Draw each snowflake:
for(f=0; f< NUMFLAKES; f++) {
display.drawBitmap(icons[f][XPOS], icons[f][YPOS], bitmap, w, h,
SSD1306_WHITE);
}
display.display(); // Show the display buffer on the screen
```

```
  delay(200);
  // Pause for 1/10 second
  // Then update coordinates of each flake...
  for(f=0; f< NUMFLAKES; f++) {
  icons[f][YPOS] += icons[f][DELTAY];
  // If snowflake is off the bottom of the screen...
  if (icons[f][YPOS] >= display.height()) {
  // Reinitialize to a random position, just off the top
  icons[f][XPOS]
  = random(1 - LOGO_WIDTH, display.width());
  icons[f][YPOS]
  = -LOGO_HEIGHT;
  icons[f][DELTAY] = random(1, 6);
  }
  }
  }
  }
```

Explicació

Aquest codi és un programa d'exemple per a Arduino que demostra les capacitats d'una pantalla OLED de 128x32 píxels (xip SSD1306 via I2C) mitjançant les biblioteques gràfiques d'Adafruit.

El programa inicialitza la pantalla i executa automàticament una bateria de proves visuals. Aquestes inclouen el dibuix de diferents formes geomètriques (línies, cercles, rectangles i triangles, tant buits com plens), la impressió de text amb diverses mides, colors i efectes de moviment (scroll), i la càrrega de petites imatges (bitmaps).

Per acabar, inicia una animació infinita on petites imatges cauen per la pantalla simulant flocs de neu. A causa d'aquest bucle continu a la configuració inicial, la funció principal `loop()` de l'Arduino queda completament buida